

Java Backend Architecture

Interview Series

Chapter 9.4

Distributed Cache – Redis

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 9.4 – Distributed Cache: Redis

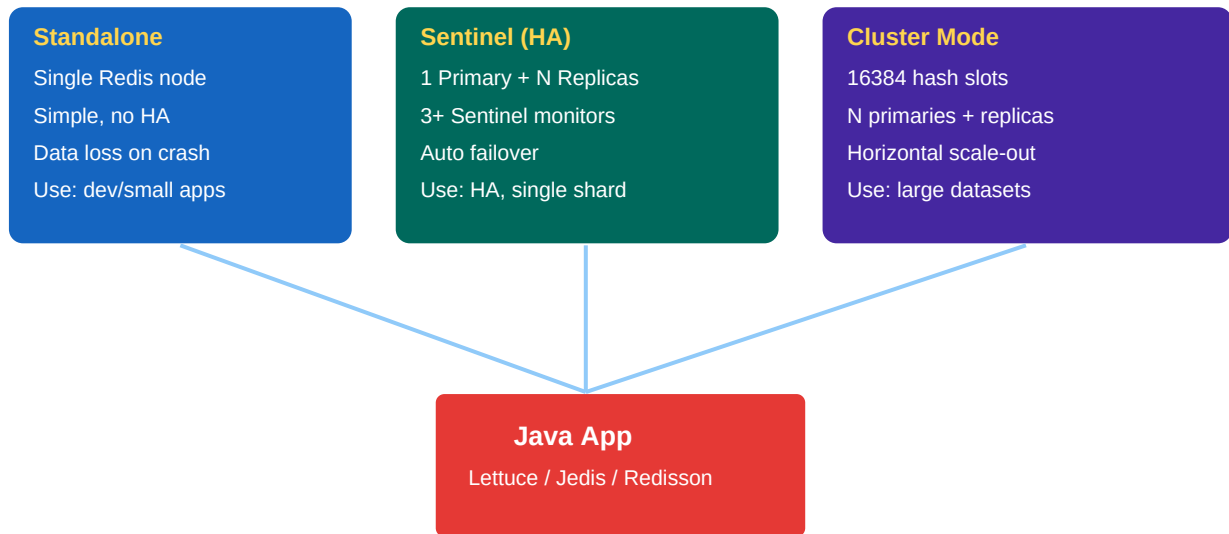
Introduction

Redis (Remote Dictionary Server) is the most widely deployed distributed cache and in-memory data store in the Java ecosystem. It operates as an external process — accessed over the network — providing a **shared cache** across all nodes of a clustered application. Redis persists data optionally, supports rich data structures (strings, hashes, lists, sets, sorted sets, streams, geo), and offers pub/sub messaging, Lua scripting, and atomic transactions via MULTI/EXEC. In Java, Redis is accessed via Lettuce (async/reactive, recommended), Jedis (sync), or Redisson (object-layer, distributed objects).

Core Concepts & Definitions

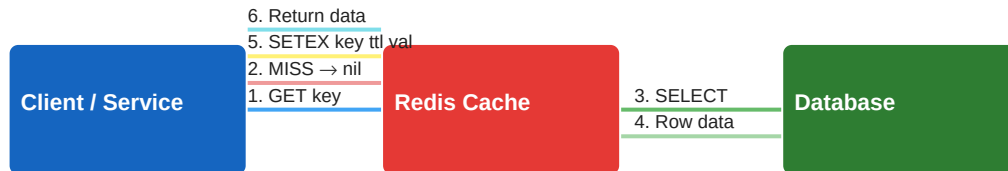
Redis	In-memory data structure store used as cache, message broker, and primary DB. Single-threaded command execution.
Lettuce	Non-blocking async Netty-based Redis client; default in Spring Data Redis; supports pipelining, pub/sub.
Jedis	Simple synchronous thread-unsafe Redis client; use JedisPool for thread safety.
Redisson	Redis client providing distributed Java objects: RMap, RList, RLock, RBucket, RAtomicLong.
SETEX / SET EX PX	Set key with TTL: SETEX key seconds value or SET key value EX 300 PX 300000.
RDB	Redis Database Backup — point-in-time snapshot saved to disk.
AOF	Append-Only File — logs every write command; more durable than RDB; fsync policy configurable.
Keyspace Notification	Redis Pub/Sub events on key operations (expire, set, del); subscribe to <code>__keyevent@*__</code> channel.
Redis Sentinel	HA solution: Sentinel processes monitor primary, elect new primary on failure, notify clients.
Redis Cluster	Sharding solution: 16384 hash slots distributed across N primary nodes, each with replicas.
Pipeline	Send multiple commands without waiting for replies; reduces RTT; non-atomic.
MULTI/EXEC	Redis transaction: queued commands executed atomically (no rollback on type error).
Lua Script	EVAL/EVALSHA: atomic multi-command operation; no interleaving; 1 script = 1 command on server.
Distributed Lock (Redlock)	Algorithm acquiring lock on N Redis nodes; majority quorum ensures safety.
Cache-Aside	App checks cache; on miss loads from DB, stores in cache; most common Redis pattern.

Redis Deployment Modes



Redis Deployment Modes – Standalone · Sentinel · Cluster

Cache-Aside Pattern



Cache-Aside Pattern (Lazy Loading): App checks cache first; on miss, loads from DB and populates cache

Also called: look-aside, read-through (with app as intermediary)

Spring Data Redis Setup

```
<!-- pom.xml --> <dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-redis</artifactId> </dependency> <!-- Lettuce is the
default client -->
```

```
# application.yml spring: data: redis: host: redis-host port: 6379 password: secret lettuce:
pool: max-active: 16 max-idle: 8 min-idle: 2 timeout: 2000ms
```

```
@Configuration public class RedisConfig { @Bean public RedisTemplate<String, Object>
redisTemplate(RedisConnectionFactory factory) { RedisTemplate<String, Object> tpl = new
RedisTemplate<>(); tpl.setConnectionFactory(factory); tpl.setKeySerializer(new
StringRedisSerializer()); tpl.setValueSerializer(new GenericJackson2JsonRedisSerializer());
tpl.setHashKeySerializer(new StringRedisSerializer()); tpl.setHashValueSerializer(new
```

```
GenericJackson2JsonRedisSerializer()); return tpl; } @Bean public RedisCacheManager
cacheManager(RedisConnectionFactory factory) { RedisCacheConfiguration cfg =
RedisCacheConfiguration.defaultCacheConfig().entryTtl(Duration.ofMinutes(30))
.serializeKeysWith(SerializationPair.fromSerializer(new StringRedisSerializer()))
.serializeValuesWith(SerializationPair.fromSerializer(new
GenericJackson2JsonRedisSerializer())) .disableCachingNullValues(); return
RedisCacheManager.builder(factory).cacheDefaults(cfg).withCacheConfiguration("products",
cfg.entryTtl(Duration.ofHours(1))).build(); }
```

Redis Data Structures

Redis Core Data Structures & Key Commands

String

SET/GET
INCR
Append

Hash

HSET/HGET
HGETALL
HDEL

List

LPUSH/RPOP
LRANGE
LLEN

Set

SADD/SMEMBERS
SINTER
SDIFF

Sorted Set

ZADD/ZRANGE
ZRANKBYScore
LEADERBOARD

Additional: Streams (XADD/XREAD), HyperLogLog (PFADD), Geo (GEOADD), Bloom Filter (BF.ADD)

```
// String – simple value redisTemplate.opsForValue().set("product:42", product,
Duration.ofMinutes(30)); Product p = (Product) redisTemplate.opsForValue().get("product:42");
// Hash – store object fields separately redisTemplate.opsForHash().put("product:42", "name",
"Widget"); redisTemplate.opsForHash().put("product:42", "price", "9.99"); Map<Object,Object>
fields = redisTemplate.opsForHash().entries("product:42"); // List – queue / recent items
redisTemplate.opsForList().leftPush("recent:views", "product:42");
redisTemplate.opsForList().trim("recent:views", 0, 99); // keep last 100 // Sorted Set –
leaderboard redisTemplate.opsForZSet().add("leaderboard", "player:1", 9500.0);
Set<ZSetOperations.TypedTuple<Object>> top10 =
redisTemplate.opsForZSet().reverseRangeWithScores("leaderboard", 0, 9); // Atomic increment
Long count = redisTemplate.opsForValue().increment("page:views:home"); // TTL management
redisTemplate.expire("product:42", Duration.ofMinutes(30)); Long ttl =
redisTemplate.getExpire("product:42", TimeUnit.SECONDS);
```

Pipelining & Transactions

```
// Pipelining: batch commands, single RTT List<Object> results =
redisTemplate.executePipelined( (RedisCallback<Object>) connection -> { StringRedisConnection
src = (StringRedisConnection) connection; src.get("k1"); src.get("k2"); src.get("k3"); return
null; }); // Transaction: MULTI/EXEC (queued, not truly isolated) redisTemplate.execute(new
SessionCallback<>() { public Object execute(RedisOperations ops) {
ops.watch("balance:user:42"); // optimistic lock ops.multi();
ops.opsForValue().decrement("balance:user:42", 100);
ops.opsForValue().increment("balance:merchant:7", 100); return ops.exec(); // null if WATCH
failed } }); // Lua Script: atomic compare-and-set String luaScript = "if redis.call('get',
KEYS[1]) == ARGV[1] then\n" + " return redis.call('set', KEYS[1], ARGV[2])\n" + "else return 0
end"; DefaultRedisScript<Long> script = new DefaultRedisScript<>(luaScript, Long.class); Long
ok = redisTemplate.execute(script, List.of("mykey"), "expected-value", "new-value");
```

Redisson: Distributed Objects & Locks

```
// Redisson config
Config config = new Config();
config.useSingleServer().setAddress("redis://redis-host:6379");
RedissonClient redisson = Redisson.create(config);
// Distributed Lock
RLock lock = redisson.getLock("order-lock:42");
try {
    boolean acquired = lock.tryLock(5, 30, TimeUnit.SECONDS);
    // wait 5s, hold 30s if (acquired) {
    // critical section – only one JVM executes this process
    order(42);
    }
} finally {
    if (lock.isHeldByCurrentThread()) lock.unlock();
}
// RMap with TTL per entry (Java Map interface)
RMap<String, Product> products = redisson.getMap("products");
products.put("p:42", product);
products.expire(Duration.ofMinutes(10));
// RAtomicLong
RAtomicLong counter = redisson.getAtomicLong("request-counter");
long val = counter.incrementAndGet();
// Rate limiter
RRateLimiter limiter = redisson.getRateLimiter("api-limiter");
limiter.trySetRate(RateType.OVERALL, 100, 1, RateIntervalUnit.SECONDS);
boolean allowed = limiter.tryAcquire();
```

Redis Cluster Configuration (Java)

```
# application.yml – Cluster mode
spring:
  data:
    redis:
      cluster:
        nodes: - redis-1:6379 - redis-2:6379 - redis-3:6379
        max-redirects: 3
lettuce:
  cluster:
    refresh: adaptive: true
# auto-discover topology changes
period: 30s
```

Keyspace Notifications for Cache Invalidation

```
# redis.conf
notify-keyspace-events "Ex"
# E=keyevent, x=expired
// Spring MessageListener for expired key events
@Component
public class RedisKeyExpiredListener implements MessageListener {
    @Autowired
    private CacheInvalidationService invalidationService;
    @Override
    public void onMessage(Message message, byte[] pattern) {
        String expiredKey = message.toString();
        // e.g., "product:42" expired → evict L1 Caffeine cache
        invalidationService.evictLocalCache(expiredKey);
    }
}
@Bean
public RedisMessageListenerContainer container(
    RedisConnectionFactory factory,
    MessageListenerAdapter listener) {
    RedisMessageListenerContainer c = new RedisMessageListenerContainer();
    c.setConnectionFactory(factory);
    c.addMessageListener(listener, new PatternTopic("__keyevent@0__:expired"));
    return c;
}
```

Interview Questions & Answers

Q1. What is Redis and why is it used as a distributed cache?

Redis is an in-memory data structure store with optional persistence. It provides sub-millisecond latency, rich data structures, TTL, pub/sub, and atomic operations. Used as distributed cache because all app nodes share one consistent view — unlike in-process caches that diverge across nodes.

Q2. What are the three Redis deployment modes?

Standalone: single node, simplest, no HA. Sentinel: primary + replicas + sentinel monitors, auto-failover. Cluster: 16384 hash slots split across N primaries each with replicas — horizontal sharding + HA.

Q3. Explain Redis hash slot distribution in Cluster mode.

Redis Cluster has 16384 hash slots. Key slot = CRC16(key) mod 16384. Each primary owns a range of slots. Clients use MOVED and ASK redirects to find the correct node. Hash tags {tag} force related keys to the same slot — required for multi-key commands in cluster mode.

Q4. What is the cache-aside pattern?

App checks cache first (GET key). On hit, returns data. On miss, queries DB, stores result in cache (SET key value EX ttl), returns data. Application controls cache population. Also called lazy loading. Disadvantage: first read always slow; stale data possible if DB updated without cache invalidation.

Q5. What is the difference between Lettuce and Jedis?

Lettuce uses Netty, is non-blocking/async, a single connection shared by all threads, supports reactive API (Flux/Mono), cluster, sentinel, SSL. Jedis is synchronous, one connection per thread, requires connection pool. Spring Data Redis defaults to Lettuce.

Q6. How does Spring @Cacheable work with Redis?

Spring's RedisCacheManager creates Cache instances backed by Redis. @Cacheable serializes the return value (default: JDK serialization; configure Jackson or Kryo) and stores it as a Redis string key with TTL. Cache key = 'cacheName::methodKey'.

Q7. What serialization format is best for Redis values in Java?

Options: JDK serialization (default, not recommended — slow, large, version-sensitive), Jackson JSON (human-readable, cross-language, moderate size), Kryo (fastest, smallest, Java-specific), Protobuf (cross-language, schema-based). Production recommendation: Jackson for simplicity or Kryo for throughput.

Q8. How do you implement a distributed lock with Redis?

Use SET key uuid NX PX 30000 (NX=only if not exists, PX=TTL ms). Release only if value matches (Lua script for atomicity). Redisson RLock automates this. Redlock algorithm uses N independent Redis nodes with quorum for distributed safety — but Martin Kleppmann notes it's unsafe without monotonic time.

Q9. What is the difference between RDB and AOF persistence?

RDB: periodic point-in-time snapshots (BGSAVE). Fast restore, but data between snapshots is lost on crash. AOF: logs every write command. More durable (fsync always/everysec/no), but larger file, slower restore. Recommended: AOF everysec for balance of durability and performance. Both can be enabled together.

Q10. What is Redis Sentinel?

Three or more Sentinel processes monitor a primary. If primary is unreachable by quorum of Sentinels, they elect a new primary (replica promotion). Clients connect to Sentinel to discover primary address. Provides HA without sharding. Not suitable when dataset > single node memory.

Q11. Explain Redis Pub/Sub and its limitations.

PUBLISH channel msg broadcasts to all subscribers. SUBSCRIBE channel registers listener. Limitations: fire-and-forget (no persistence), if subscriber is down it misses messages, no consumer groups, no replay. For reliable messaging use Redis Streams (XADD/XREAD) instead.

Q12. What is pipelining in Redis?

Pipelining sends multiple commands to Redis without waiting for each reply. Reduces network RTT from N to 1 for N commands. Not atomic — server processes commands normally but client batches socket writes. Use for bulk loads. Different from MULTI/EXEC which is atomic.

Q13. How does Redis MULTI/EXEC work?

MULTI starts a transaction queue. Commands are queued (not executed). EXEC atomically executes all queued commands. DISCARD cancels. WATCH key implements optimistic locking — if watched key changes before EXEC, transaction is aborted (EXEC returns nil). Note: no rollback — errors in execution don't abort others.

Q14. What is a Lua script in Redis and why use it?

EVAL script numkeys key1 ... arg1 ... executes a Lua script atomically on the server. No interleaving from other commands. Used for atomic compare-and-swap, conditional operations, or multiple-step operations that must be atomic. EVALSHA uses pre-loaded script hash for efficiency.

Q15. Explain the Redlock algorithm.

To acquire a lock across N Redis nodes: set key with NX and TTL on each node, measuring elapsed time. Lock acquired if $>N/2$ nodes succeed and elapsed time $< \text{TTL}$. Release by Lua-delete on all nodes. Provides safety if fewer than $N/2$ nodes fail simultaneously. Controversial: Kleppmann argues it needs fencing tokens for true distributed safety.

Q16. What are the memory eviction policies in Redis?

noeviction (default — error on OOM), allkeys-lru, allkeys-lfu, volatile-lru, volatile-lfu, allkeys-random, volatile-random, volatile-ttl. For cache use: allkeys-lfu (best hit rate by frequency) or allkeys-lru. volatile-* only evicts keys with TTL set.

Q17. How does Redis Cluster handle cross-slot multi-key operations?

MGET/MSET/MULTI/EXEC across different slots are not supported in cluster mode. Solutions: 1) Hash tags — {tag}:key1 and {tag}:key2 hash to same slot. 2) Store all related data in one key (Hash). 3) Use client-side logic to group by slot. 4) Use a single node for multi-key operations (defeats sharding).

Q18. What is the difference between EXPIRE and EXPIREAT?

EXPIRE key seconds: sets relative TTL from now. EXPIREAT key timestamp: sets absolute Unix timestamp for expiry. PEXPIRE/PEXPIREAT for milliseconds. TTL key returns remaining seconds (-1=persistent, -2=no key). PERSIST key removes TTL.

Q19. How does Redis handle memory limits?

maxmemory config sets memory cap. When reached, eviction policy applies. Monitor with INFO memory: used_memory, used_memory_rss, mem_fragmentation_ratio. Fragmentation > 1.5 indicates fragmentation; MEMORY PURGE (Redis 4+) can defragment.

Q20. What is a hot key problem in Redis and how do you solve it?

A single key receives disproportionate traffic (e.g., viral product page). Single Redis slot bottleneck. Solutions: key sharding (product:42:shard:{0-9} with random selection), local L1 cache (Caffeine) in front of Redis with short TTL, Redis Cluster read replicas with READONLY command, application-level fan-out.

Q21. What is the big key problem in Redis?

A key with a very large value (e.g., 10MB string, hash with 1M fields). Problems: blocks Redis single-threaded processing, large network transfer, slow eviction. Detect: redis-cli --bigkeys. Solutions: split into multiple smaller keys, use HSCAN/SSCAN for iteration, compress values, move to separate storage.

Q22. How do you implement rate limiting with Redis?

Fixed window: INCR key, EXPIRE if new, reject if $> \text{limit}$. Sliding window: Sorted set with ZADD (timestamp as score), ZREMRANGEBYSCORE old entries, ZCARD for count. Token bucket: Lua script decrementing token key. Redisson RRateLimiter implements this natively.

Q23. What is Redis Streams and how does it differ from Pub/Sub?

Redis Streams (XADD/XREAD/XREADGROUP) is a persistent, replayable, consumer-group-based log. Unlike Pub/Sub: messages are persisted, consumers can replay from offset, multiple consumer groups with independent offsets, XACK for acknowledgment. Similar to Kafka but simpler, not distributed.

Q24. How do you monitor Redis health in production?

redis-cli INFO: memory (used_memory, fragmentation), stats (keyspace_hits, keyspace_misses, hit_rate), replication (role, connected_slaves), clients (connected_clients, blocked_clients). Alerts: latency $> 1\text{ms}$ (LATENCY MONITOR), hit_rate < 0.9 , memory $> 80\%$ maxmemory.

Q25. What is the Redis Object Encoding and why does it matter?

Redis uses compact encodings for small objects: listpack (formerly ziplist) for small hashes/lists/zsets, intset for integer sets, embstr for short strings (<44 bytes). These save memory. When size exceeds threshold (hash-max-listpack-entries, etc.), encoding upgrades to hashtable/skiplist — more memory. Tuning thresholds can significantly reduce memory footprint.

Q26. Describe a two-level cache architecture using Caffeine and Redis.

L1: Caffeine (in-process, nanosecond, short TTL e.g. 1 min). L2: Redis (shared, millisecond, longer TTL e.g. 30 min). On read: check L1, on miss check L2, on miss load from DB. Write to both. On L2 hit, back-fill L1. Invalidation: Redis Keyspace Notification or Kafka event evicts L1 on all nodes.

Q27. What is the Spring Data Redis ReactiveRedisTemplate?

ReactiveRedisTemplate uses Lettuce's reactive Netty driver. All operations return Mono or Flux. Enables non-blocking cache access in WebFlux applications. Setup similar to RedisTemplate but with ReactiveRedisConnectionFactory and reactive serializers.

Q28. How do you handle Redis connection failures gracefully?

Lettuce auto-reconnects. Configure connection timeout and command timeout. Use Circuit Breaker (Resilience4j) around Redis calls — on open circuit, fall through to DB. Implement fallback: if Redis is down, serve from DB directly without caching. Log metrics: redis_connection_errors_total.

Q29. What is Redis SCAN and why is it preferred over KEYS?

KEYS pattern iterates all keys in $O(N)$ — blocks single-threaded Redis. SCAN cursor count pattern is incremental: returns a cursor and partial results per call; non-blocking. Use SCAN in production for key enumeration. HSCAN/SSCAN/ZSCAN for field iteration within a key.

Q30. Explain the difference between optimistic and pessimistic locking in Redis.

Pessimistic: use SETNX (SET NX) to acquire a lock key; hold it during operation; delete after. Blocks other clients. Optimistic: WATCH key → read → MULTI → modify → EXEC. If watched key changed, EXEC returns nil. Retry logic needed. Pessimistic suits short critical sections; optimistic suits read-heavy with rare conflicts.

Q31. How do you implement cache warm-up for Redis?

Startup: @PostConstruct or ApplicationReadyEvent listener bulk-loads critical data. Use Redis pipeline for efficiency. Alternative: export/import RDB on startup, or replay a warm-up event log. For microservices: dedicated warm-up job reads DB and populates Redis before routing traffic.

Q32. What are Redis Sentinel quorum and failover thresholds?

Quorum: minimum Sentinels that must agree primary is down to start failover. Typically $(N+1)/2$ of N Sentinels. down-after-milliseconds: time primary must be unresponsive before marked down. failover-timeout: max time for failover. min-slaves-to-write: primary accepts writes only if N replicas connected.

Q33. What is the impact of Redis persistence on latency?

AOF fsync always: fsync on every write — safest but adds ~1ms latency. AOF fsync everysec (default): fsync once per second — max 1s data loss, negligible latency impact. RDB BGSAVE: fork + copy-on-write — parent latency spike during fork on large datasets. Recommendation: AOF everysec for caching workloads.

Q34. How do you implement a session store with Redis and Spring Session?

Add spring-session-data-redis dependency. @EnableRedisHttpSession. Sessions stored as Redis Hashes with configurable TTL. Horizontal scaling works automatically — any node reads from Redis. Cookie JSESSIONID maps to Redis key spring:session:sessions:{id}.

Q35. What is the Redis hash-slots migration process?

CLUSTER SETSLOT slot MIGRATING src → CLUSTER SETSLOT slot IMPORTING dst → MIGRATE key → CLUSTER SETSLOT slot NODE dst. During migration, clients use ASK redirect. Automatic rebalancing available in Redis Enterprise and `redis-cli --cluster rebalance`.

Q36. Describe a cache invalidation strategy using Redis Pub/Sub.

On entity update: service A publishes 'invalidate:product:42' to Redis channel 'cache-invalidation'. All services subscribe and evict local Caffeine entries matching the key. Simple, low coupling. Limitation: if subscriber is down during publish, it misses the event. Use Streams for at-least-once.

Q37. What is write-behind caching and how is it implemented with Redis?

Write to Redis immediately; persist to DB asynchronously. Implementation: write to Redis List/Stream, background worker (consumer group) reads and flushes to DB in batches. Reduces write latency but risks data loss if Redis fails before flush. Requires Redis persistence (AOF) for durability.

Q38. How does Lettuce handle Redis Cluster topology changes?

Lettuce's `ClusterTopologyRefreshOptions` enables periodic or adaptive topology refresh. Adaptive refresh reacts to MOVED/ASK redirects and timeouts. When new nodes are added or slots migrate, Lettuce refreshes its partition map and routes future commands correctly without restart.

Q39. What is the Redis Object TTL precision and minimum TTL?

Redis TTL precision is 1ms since Redis 2.6 (`PEXPIRE/PTTL` for milliseconds). Minimum TTL is 1ms (`PEXPIRE key 1`). Keys expire lazily (accessed) and actively (background task samples 20 keys/sec per database, deletes if expired). Large volumes of expiring keys can cause latency spikes.

Q40. How do you ensure idempotency in cache writes with Redis?

Use SET key value NX (only set if not exists) for insert-idempotency. Use WATCH + MULTI/EXEC or Lua script for compare-and-set idempotency. Include a version/etag in the cached object and reject writes where incoming version < stored version.

Q41. What are the security best practices for Redis?

1) requirepass with strong password. 2) bind to localhost or private network only. 3) Rename dangerous commands (FLUSHALL, CONFIG, DEBUG) in `redis.conf` with `rename-command`. 4) Enable TLS (Redis 6+). 5) ACL (Redis 6+): per-user command and key permissions. 6) Never expose Redis port publicly.

Q42. What is Redis Cluster's handling of node failure and data loss?

If a primary fails before its replica has replicated all writes (replication lag), those writes are lost. `cluster-require-full-coverage: yes` stops writes if any slot has no healthy primary. `cluster-node-timeout`: max time node can be unreachable before considered failed. Replica promotion adds latency during failover (seconds).

Q43. How do you tune Redis for maximum throughput?

1) Use pipelining to batch commands. 2) Disable persistence (no AOF/RDB) for pure cache. 3) Use appropriate data structures (avoid big lists). 4) Enable lazy-freeing (`lazyfree-lazy-eviction yes`). 5) Use Lettuce connection pooling with appropriate pool size. 6) Set `tcp-backlog 511`, `hz 100` for high-connection servers. 7) Disable swap: ensure `maxmemory set`.

Q44. What is the difference between FLUSHDB and FLUSHALL?

FLUSHDB: deletes all keys in the current database (default DB 0). FLUSHALL: deletes all keys in all databases. Both are synchronous by default (blocking); add `ASYNC` for background execution (Redis 4+). Never run without `ASYNC` in production — large DBs block Redis for seconds.

Q45. How do you use Redis for implementing a bloom filter for cache optimization?

RedisBloom module: BF.ADD filter key to add, BF.EXISTS filter key to check. In front of Redis cache: check bloom filter — if says absent, skip Redis lookup (definitely not cached). False positives (item says present but isn't) result in cache miss, acceptable. Reduces Redis load for lookup of non-existent keys (e.g., negative caching).

Q46. Explain Redis lazy expiration vs active expiration.

Lazy expiration: key deleted when accessed and found expired. Active expiration: background task every 100ms samples 20 keys per DB with TTL; deletes expired ones; if >25% expired, repeats. With many short-TTL keys, memory may not be freed quickly — use smaller db size or increase hz.

Q47. How do you implement geo-distributed caching with Redis?

Redis Enterprise Active-Active (CRDT-based) geo-replication: each region has a full Redis cluster; writes are replicated asynchronously; conflicts resolved via CRDT rules. Open-source alternative: Redis Sentinel with cross-datacenter replication (async, eventual). Applications use local region Redis for reads; cross-region writes may have replication lag.

Q48. What is the role of the Redis client-side caching (tracking) feature?

Redis 6+ server-side tracking: client sends CLIENT TRACKING on. Redis records which keys each client read. When a key changes, Redis sends an invalidation message to the tracking client via a Pub/Sub channel. Client invalidates its in-memory copy. Enables consistent client-side cache without polling.

Q49. How would you design a cache for a multi-tenant SaaS application using Redis?

Key format: tenant:{tenantId}:{entity}:{id}. Separate TTLs per tenant tier (premium=longer). Use Redis 6 ACL to restrict each tenant's access to their key prefix. Monitor per-tenant memory with INFO keypace and custom key prefix scanning. Rate-limit per-tenant Redis operations to prevent noisy-neighbor problems. Consider separate Redis databases or clusters for strict isolation.